

React Hooks

The Complete Guide

All 10 hooks every React developer must know —
taught from scratch with real-world analogies, code, and interview Q&A.

1. `useState` — State management
2. `useEffect` — Side effects & lifecycle
3. `useMemo` — Expensive computations
4. `useCallback` — Stable function references
5. `useContext` — Global state without prop drilling
6. `useReducer` — Complex state logic
7. `useRef` — DOM access & mutable values
8. `useImperativeHandle` — Custom ref API
9. `useLayoutEffect` — Synchronous DOM mutations
10. `useDebugValue` — DevTools labels for custom hooks

Prepared for Akshay — Interview Prep Series

#1

useState

Add memory to your component — lets it remember values between renders.

What is useState?

In plain English: **useState** gives your component a variable that React watches. When you change it, React re-renders the component with the new value. Without useState, any variable you create inside a function component would be reset every time the component re-renders.

Real-World Analogy

Think of a scoreboard at a cricket match. The score (state) is displayed to everyone (rendered on screen). When a run is scored, someone behind the board flips the numbers (setState). The board then shows the updated score (re-render). You never erase the whole board and redraw — you just update the changed part.

Syntax

```
const [value, setValue] = useState(initialValue);  
// value → current state value  
// setValue → function to update state  
// initialValue → starting value (runs only on first render)
```

Beginner — Basic Counter

The simplest use — track a single number and update it when a button is clicked.

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return (  
    <div>  
      <p>Count: {count}</p>  
      <button onClick={() => setCount(count + 1)}>+1</button>  
    </div>  
  );  
}
```

■ ■ **Key Rule:** Never mutate state directly. `count = 5` won't work — React won't know to re-render. Always use the setter function.

useState with Different Data Types

```
// String
const [name, setName] = useState("");

// Boolean
const [isOpen, setIsOpen] = useState(false);

// Array
const [items, setItems] = useState([]);

// Object
const [user, setUser] = useState({ name: "", age: 0 });

// null (data not yet loaded)
const [data, setData] = useState(null);
```

Intermediate — Updating Objects & Arrays (Immutably)

React uses **reference comparison** (`Object.is`) to decide if state changed. If you mutate an object/array directly, the reference stays the same, and React skips the re-render. You must create a **NEW** object/array every time.

✗ Wrong

```
// Mutating directly – React won't re-render!
const handleChange = (e) => {
  user.name = e.target.value;
  setUser(user); // same reference
};
```

✓ Correct

```
// Creating new object – React sees the change
const handleChange = (e) => {
  setUser({
    ...user,
    name: e.target.value,
  }); // new reference
};
```

Array operations (immutable patterns)

```
// ADD item
setItems(prev => [...prev, newItem]);

// REMOVE item
setItems(prev => prev.filter(item => item.id !== id));

// UPDATE item
setItems(prev => prev.map(item =>
  item.id === id ? { ...item, done: true } : item
));

// These all create NEW arrays – React detects the change.
```

Expert — Functional Updates & Lazy Initialization

Functional Updates (Stale Closure Fix)

When your new state depends on the old state, ALWAYS use the function form. Otherwise you'll hit the **stale closure** bug — one of the most common React interview gotchas.

✗ Wrong

```
// BUG: stale closure
const handleClick = () => {
  setCount(count + 1); // reads count = 0
  setCount(count + 1); // still reads count = 0
  setCount(count + 1); // still reads count = 0
  // Result: count goes from 0 to 1 (not 3!)
};
```

✓ Correct

```
// CORRECT: functional update
const handleClick = () => {
  setCount(c => c + 1); // c = 0 → 1
  setCount(c => c + 1); // c = 1 → 2
  setCount(c => c + 1); // c = 2 → 3
  // Result: count goes from 0 to 3
};
```

Lazy Initialization

If computing the initial state is expensive (e.g., reading from localStorage, parsing JSON), pass a **function** to useState. It runs only on the first render.

✗ Wrong

```
// Runs on EVERY render (wasteful)
const [data, setData] = useState(
  JSON.parse(localStorage.getItem("data"))
);
```

✓ Correct

```
// Runs only on FIRST render (lazy)
const [data, setData] = useState(() =>
  JSON.parse(localStorage.getItem("data"))
);
```

React 18 Automatic Batching

In React 18+, multiple setState calls in the same event handler (or even setTimeout, Promise, etc.) are **batched** into a single re-render. This is automatic — you don't need to do anything.

```
function handleClick() {
  setCount(c => c + 1);
  setName("Akshay");
  setAge(25);
  // Only ONE re-render happens here, not three
}
```

Interview Q&A

Q: What happens if you call useState conditionally?

A: React throws an error. Hooks must be called in the same order every render — never inside if/else, loops, or nested functions. This is the **Rules of Hooks**.

Q: What is the difference between setState(newValue) and setState(prev => newValue)?

A: The first reads the current value from the closure (can be stale). The second receives the actual latest state as an argument — always safe for dependent updates.

Q: Does `setState` trigger an immediate re-render?

A: No. `setState` schedules a re-render. React batches updates and processes them asynchronously. The new state is available in the NEXT render, not immediately after calling `setState`.

Q: What if you call `setState` with the same value?

A: React bails out — it skips the re-render (using `Object.is` comparison). For objects/arrays, you must pass a new reference for React to detect the change.

#2

useEffect

Run side effects after render — API calls, subscriptions, timers, DOM manipulation.

What is useEffect?

In plain English: **useEffect** tells React 'after you finish rendering this component, run this code.' It's for things that happen OUTSIDE the component's render — fetching data, setting up a timer, updating the document title, subscribing to an event.

Real-World Analogy

Imagine a waiter at a restaurant. Their main job is taking orders and serving food (render). But AFTER they serve each table, they also need to wipe the table, refill water, etc. (side effects). useEffect is like saying 'after you finish serving, do this cleanup task.'

Syntax & The 3 Modes

```
// Mode 1: Run after EVERY render (no dependency array)
useEffect(() => {
  console.log("I run after every render");
});

// Mode 2: Run ONCE on mount (empty dependency array)
useEffect(() => {
  console.log("I run only once, on mount");
}, []);

// Mode 3: Run when specific values change
useEffect(() => {
  console.log("I run when 'count' changes");
}, [count]);
```

■ **Mental Model:** The dependency array answers: 'What does this effect depend on?' If any dependency changes between renders, the effect re-runs.

The Cleanup Function

If your effect creates something that needs to be cleaned up (timer, subscription, event listener), return a function from `useEffect`. React calls this cleanup function before re-running the effect and when the component unmounts.

```
useEffect(() => {
```

```
// SETUP – runs after render
const timer = setInterval(() => console.log("tick"), 1000);
// CLEANUP – runs before next effect & on unmount
return () => clearInterval(timer);
}, []);

// Without cleanup, you get a MEMORY LEAK – the interval
// keeps running even after the component is removed from DOM.
```

Beginner — Fetching Data

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    let cancelled = false; // race-condition guard
    async function fetchUser() {
      setLoading(true);
      const res = await fetch(`/api/users/${userId}`);
      const data = await res.json();
      if (!cancelled) {
        setUser(data);
        setLoading(false);
      }
    }
    fetchUser();
    return () => { cancelled = true; }; // cleanup
  }, [userId]); // re-fetch when userId changes
  if (loading) return <p>Loading...</p>;
  return <h1>{user.name}</h1>;
}
```

■ ■ **Why the cancelled flag?** If `userId` changes quickly ($A \rightarrow B$), `fetch A` might return AFTER `fetch B`. Without the flag, you'd show stale data from A. The cleanup sets `cancelled = true` for the old effect.

Intermediate — Event Listeners & Resize

```
function WindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight,
  });
  useEffect(() => {
```

```
const handleResize = () => setSize({
  width: window.innerWidth,
  height: window.innerHeight,
});
window.addEventListener("resize", handleResize);
return () => window.removeEventListener("resize", handleResize);
}, []); // empty = mount/unmount only
return <p>{size.width} x {size.height}</p>;
}
```

Expert — Common Mistakes

✗ Wrong

```
// INFINITE LOOP – object in deps
useEffect(() => {
  fetch("/api").then(r => r.json())
    .then(setData);
}, [{ page: 1 }]);
// New object ref every render
// → effect re-runs → re-render → loop
```

✓ Correct

```
// CORRECT – primitive in deps
const page = 1;
useEffect(() => {
  fetch(`/api?page=${page}`)
    .then(r => r.json())
    .then(setData);
}, [page]);
// Number comparison is stable
```

✗ Wrong

```
// WRONG: async function directly
useEffect(async () => {
  const res = await fetch("/api");
  const data = await res.json();
  setData(data);
}, []);
// useEffect expects a cleanup fn or
// undefined, NOT a Promise
```

✓ Correct

```
// CORRECT: async INSIDE the effect
useEffect(() => {
  async function load() {
    const res = await fetch("/api");
    const data = await res.json();
    setData(data);
  }
  load();
}, []);
```

Execution Order (Interview Favourite)

```
function App() {
  console.log("1. Render"); // runs first
  useEffect(() => {
    console.log("3. Effect"); // runs AFTER paint
    return () => console.log("4. Cleanup"); // runs before next effect
  });
  console.log("2. Still rendering"); // synchronous
  return <div>Hello</div>;
}
```



```
}  
  
// Output on mount: 1, 2, 3  
// Output on update: 1, 2, 4 (cleanup old), 3 (run new)  
// Output on unmount: 4 (cleanup)
```

Interview Q&A

Q: What's the difference between `useEffect` and `useLayoutEffect`?

A: `useEffect` runs AFTER the browser paints the screen (async). `useLayoutEffect` runs BEFORE the browser paints (sync). Use `useLayoutEffect` only when you need to measure/mutate the DOM before the user sees it.

Q: Can you have multiple `useEffect` calls in one component?

A: Yes! It's actually encouraged. Separate effects by concern — one for data fetching, another for event listeners, etc. This is better than one giant effect with everything inside.

Q: Why shouldn't you make the `useEffect` callback async?

A: `useEffect` expects you to return either nothing (undefined) or a cleanup function. An async function returns a Promise — React can't use a Promise as a cleanup. Wrap the async logic in an inner function instead.

#3

useMemo

Cache the result of an expensive calculation — only recompute when dependencies change.

What is useMemo?

In plain English: **useMemo** says 'remember the result of this computation. Only recalculate it if the inputs change.' It's React's way of avoiding repeated expensive work.

Real-World Analogy

*Imagine you calculate your monthly expenses in a spreadsheet. You don't re-add every bill every time you open the sheet — you only recalculate when a bill amount changes. **useMemo** is that spreadsheet cache.*

Syntax

```
const memoizedValue = useMemo(() => {  
  return expensiveCalculation(a, b);  
}, [a, b]); // only recomputes when a or b changes
```

When to Use (and When NOT To)

- **USE when:** Filtering/sorting large lists, heavy math, creating complex derived objects.
- DON'T USE when:** Simple calculations, string concatenation, basic arithmetic. **useMemo** itself has overhead — the memoization must be cheaper than the calculation.

Beginner — Filtering a Large List

```
function ProductList({ products, searchTerm }) {  
  // Without useMemo: filters 10,000 products on EVERY keystroke  
  // even if only an unrelated state (like a modal) changed  
  const filtered = useMemo(() => {  
    console.log("Filtering..."); // check how often this runs  
    return products.filter(p =>  
      p.name.toLowerCase().includes(searchTerm.toLowerCase())  
    );  
  }, [products, searchTerm]);  
  // Only re-filters when products or searchTerm actually changes  
  return filtered.map(p => <div key={p.id}>{p.name}</div>);  
}
```

```
}
```

Expert — Referential Equality

useMemo also helps with **referential equality**. If you pass a new object/array as a prop or dependency on every render, child components re-render unnecessarily.

✗ Wrong

```
// New object every render
function Parent() {
  const config = { theme: "dark", lang: "en" };
  // config is a NEW object every render
  // Child re-renders even if values same
  return <Child config={config} />;
}
```

✓ Correct

```
// Stable reference with useMemo
function Parent() {
  const config = useMemo(() => ({
    theme: "dark", lang: "en"
  }), []); // same reference across renders
  return <Child config={config} />;
}
```

Interview Q&A

Q: What's the difference between useMemo and useCallback?

A: useMemo caches the **RESULT** of a function (a value). useCallback caches the **FUNCTION ITSELF** (a reference). useMemo(() => fn, deps) returns the computed value. useCallback(fn, deps) returns the function fn.

Q: Does useMemo guarantee the value won't be recomputed?

A: No. React may throw away the cached value to free memory (e.g., offscreen components). useMemo is a performance **HINT**, not a semantic guarantee. Your code must work correctly even without it.

#4

useCallback

Cache a function definition — prevent unnecessary re-creation on every render.

What is useCallback?

In plain English: every time a component re-renders, all functions inside it are re-created. Usually that's fine. But if you pass a function as a prop to a memoized child (`React.memo`), the child will re-render because it sees a 'new' function. **useCallback** says 'keep the same function reference unless my dependencies change.'

Real-World Analogy

*Imagine your boss gives you the same instruction sheet every Monday. If it's word-for-word identical, you'd rather they just say 'same as last week' instead of printing a new copy. **useCallback** is that 'same as last week' — same function reference, no new copy.*

Syntax

```
const memoizedFn = useCallback(() => {
  doSomething(a, b);
}, [a, b]); // new function only when a or b changes

// Equivalent to:
const memoizedFn = useMemo(() => {
  return () => doSomething(a, b);
}, [a, b]);
```

When It Matters

```
// WITHOUT useCallback – Child re-renders on every Parent render
function Parent() {
  const [count, setCount] = useState(0);
  const handleClick = () => console.log("clicked");
  // handleClick is a NEW function every render
  return <MemoizedChild onClick={handleClick} />;
}

// WITH useCallback – Child skips unnecessary re-renders
function Parent() {
  const [count, setCount] = useState(0);
  const handleClick = useCallback(() => {
    console.log("clicked");
```

```
}, [])); // same reference across renders
return <MemoizedChild onClick={handleClick} />;
}

const MemoizedChild = React.memo(({ onClick }) => {
  console.log("Child rendered");
  return <button onClick={onClick}>Click</button>;
});
```

■ ■ **Rule of thumb:** `useCallback` only helps if the child is wrapped in `React.memo` (or if the function is a dependency of `useEffect/useMemo`). Without `React.memo` on the child, `useCallback` is pointless overhead.

Interview Q&A

Q: When should you NOT use `useCallback`?

A: When the child component is NOT memoized with `React.memo`, when the function is not used as a dependency of another hook, or when the component is simple and cheap to re-render. Premature memoization adds complexity without benefit.

Q: Can `useCallback` prevent re-renders by itself?

A: No! `useCallback` only stabilizes the function reference. The CHILD must be wrapped in `React.memo` to actually skip re-rendering. `useCallback` without `React.memo` is like locking a door that has no walls.

#5

useContext

Access shared data anywhere in the tree — without passing props through every level.

What is useContext?

In plain English: **useContext** solves **prop drilling** — the problem where you pass data through 5 levels of components just to reach a deeply nested child. Instead, you create a 'broadcast channel' (Context) and any component can tune in directly.

Real-World Analogy

Prop drilling is like passing a note through every student in a row to reach the last one. Context is like the teacher making an announcement — everyone hears it directly, no passing needed.

3-Step Pattern

```
// STEP 1: Create the Context
const ThemeContext = createContext("light"); // default value

// STEP 2: Provide — wrap the tree with a Provider
function App() {
  const [theme, setTheme] = useState("dark");
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Header />
      <Main />
    </ThemeContext.Provider>
  );
}

// STEP 3: Consume — use the context in any child
function Header() {
  const { theme, setTheme } = useContext(ThemeContext);
  return (
    <header className={theme}>
      <button onClick={() => setTheme(t => t === "dark" ? "light" : "dark")}>
        Toggle: {theme}
      </button>
    </header>
  );
}
```

Expert — Performance Trap & Fix

When a Context value changes, **every** component using that context re-renders — even if they only use one field from it. This is the #1 performance issue with Context.

✗ Wrong

```
// ALL consumers re-render when ANY value
changes
const AppContext = createContext();
function App() {
  const [user, setUser] = useState(null);
  const [theme, setTheme] = useState("dark");
  return (
    <AppContext.Provider value={{ user, theme
    }}>
    <Page />
    </AppContext.Provider>
  );
}
// Header only needs theme but re-renders
// when user changes too
```

✓ Correct

```
// SPLIT contexts by update frequency
const UserContext = createContext();
const ThemeContext = createContext();
function App() {
  const [user, setUser] = useState(null);
  const [theme, setTheme] = useState("dark");
  return (
    <UserContext.Provider value={user}>
    <ThemeContext.Provider value={theme}>
    <Page />
    </ThemeContext.Provider>
    </UserContext.Provider>
  );
}
// Now they update independently
```

Interview Q&A

Q: Is Context a replacement for Redux / Zustand?

A: For simple global state (theme, auth, locale) — yes. For complex state with frequent updates, heavy derived data, or middleware — no. Context re-renders ALL consumers on every change; Redux/Zustand have fine-grained subscriptions.

Q: What happens if there's no Provider above a useContext call?

A: The component receives the DEFAULT value passed to createContext(defaultValue). No error is thrown. This is useful for testing components in isolation.

#6

useReducer

Manage complex state with predictable action-based updates — like a mini Redux.

What is useReducer?

In plain English: **useReducer** is `useState`'s big brother. Instead of calling `setState` directly, you send 'actions' (descriptions of what happened) to a 'reducer' function that decides how to update state. Best for state that has multiple sub-values or complex update logic.

Real-World Analogy

Think of a bank account. You don't directly change your balance — you submit transactions (deposit, withdraw, transfer). The bank (reducer) processes each transaction and computes the new balance. The transaction slip is the 'action'; the bank's logic is the 'reducer.'

Syntax

```
const [state, dispatch] = useReducer(reducerFn, initialState);

// reducerFn → (currentState, action) => newState
// dispatch → function to send actions
// actions → { type: "ACTION_NAME", payload: data }
```

Complete Example — Shopping Cart

```
const initialState = { items: [], total: 0 };

function cartReducer(state, action) {
  switch (action.type) {
    case "ADD_ITEM": {
      const items = [...state.items, action.payload];
      return { items, total: items.reduce((s, i) => s + i.price, 0) };
    }
    case "REMOVE_ITEM": {
      const items = state.items.filter(i => i.id !== action.payload);
      return { items, total: items.reduce((s, i) => s + i.price, 0) };
    }
    case "CLEAR":
      return initialState;
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}
```



```

}

function Cart() {
  const [state, dispatch] = useReducer(cartReducer, initialState);
  return (
    <div>
      <button onClick={() => dispatch({
        type: "ADD_ITEM",
        payload: { id: 1, name: "Shirt", price: 499 }
      })}>
        Add Shirt
      </button>
      <p>Total: {state.total}</p>
    </div>
  );
}

```

useState vs useReducer — When to Choose

Criteria	useState	useReducer
Simple values (boolean, string, number)	✓ Best	Overkill
Object with 3+ fields updated together	Gets messy	✓ Best
Complex update logic (conditionals)	Hard to read	✓ Best
State transitions depend on previous state	Works	✓ Cleaner
Shared update logic (multiple dispatch sites)	Duplicated	✓ Centralized
Testing the update logic in isolation	Harder	✓ Pure function

Interview Q&A

Q: Can useReducer replace Redux?

A: For component-level or small app state — yes. But useReducer doesn't provide middleware, devtools, or cross-component subscriptions out of the box. For large apps, Redux Toolkit or Zustand still have advantages.

#7

useRef

Hold a mutable value that persists across renders — without causing re-renders.

What is useRef?

In plain English: **useRef** gives you a box (`.current`) that holds any value. Unlike `useState`, changing `ref.current` does NOT trigger a re-render. It has two main uses: (1) accessing DOM elements directly, and (2) storing mutable values that persist across renders.

Real-World Analogy

useState is like writing on a whiteboard that everyone can see (render). useRef is like writing in your personal notebook — you can update it anytime, but nobody else knows until they check.

Use Case 1: DOM Access

```
function SearchBar() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current.focus(); // auto-focus on mount
  }, []);

  return <input ref={inputRef} placeholder="Search..." />;
}
```

Use Case 2: Mutable Value (no re-render)

```
function Stopwatch() {
  const [time, setTime] = useState(0);
  const intervalRef = useRef(null); // persists across renders

  const start = () => {
    intervalRef.current = setInterval(() => {
      setTime(t => t + 1);
    }, 1000);
  };

  const stop = () => clearInterval(intervalRef.current);

  return (
    <div>
      <p>{time}s</p>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
    </div>
  );
}
```

```

</div>
);
}
// intervalRef.current changes but no re-render is triggered

```

Use Case 3: Previous Value

```

function usePrevious(value) {
  const ref = useRef();
  useEffect(() => {
    ref.current = value; // update AFTER render
  });
  return ref.current; // returns value from PREVIOUS render
}

// Usage
function Counter() {
  const [count, setCount] = useState(0);
  const prevCount = usePrevious(count);
  return <p>Now: {count}, Before: {prevCount}</p>;
}

```

useState vs useRef

✗ Wrong

```

// useState → triggers re-render
const [count, setCount] = useState(0);
setCount(1); // component re-renders
// Use for: anything the UI depends on

```

✓ Correct

```

// useRef → NO re-render
const countRef = useRef(0);
countRef.current = 1; // no re-render
// Use for: timer IDs, previous values,
// DOM refs, render counts, etc.

```

Interview Q&A

Q: Can you update a ref during render?

A: You shouldn't. Updating `ref.current` during render can lead to inconsistent behavior because React may render a component multiple times (Strict Mode, concurrent features). Update refs in `useEffect` or event handlers instead.

Q: What's the difference between `useRef` and `createRef`?

A: `useRef` returns the SAME ref object on every render. `createRef` creates a NEW ref each render — it's meant for class components. In function components, always use `useRef`.

#8

useImperativeHandle

Customize what the parent gets when it uses a ref on your component.

What is useImperativeHandle?

In plain English: when a parent passes a ref to your component (via `forwardRef`), by default they get the raw DOM node. **useImperativeHandle** lets you restrict or customize what the parent can access — you expose a custom API instead of the full DOM.

Real-World Analogy

Think of a car's dashboard. You get a steering wheel, pedals, and buttons (the API). You DON'T get direct access to the engine, fuel injectors, or transmission. `useImperativeHandle` is the dashboard — it exposes only what the user needs.

Syntax

```
useImperativeHandle(ref, () => ({
  // methods/properties to expose to the parent
  focus: () => { ... },
  scrollTop: () => { ... },
}), [dependencies]);
```

Complete Example — Custom Input

```
import { forwardRef, useRef, useImperativeHandle } from "react";
const FancyInput = forwardRef((props, ref) => {
  const inputRef = useRef(null);
  // Expose ONLY these methods to the parent
  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current.focus(),
    clear: () => {
      inputRef.current.value = "";
      inputRef.current.focus();
    },
    getValue: () => inputRef.current.value,
  }));
  return <input ref={inputRef} {...props} />;
});
// Parent usage
```

```
function Form() {
  const inputRef = useRef(null);
  return (
    <div>
      <FancyInput ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}>Focus</button>
      <button onClick={() => inputRef.current.clear()}>Clear</button>
      { /* inputRef.current.style → undefined (not exposed!) */ }
    </div>
  );
}
```

■ **React 19 Update:** In React 19+, you can pass ref as a regular prop — no forwardRef needed. But useImperativeHandle still works the same way for customizing the exposed API.

Interview Q&A

Q: When would you use useImperativeHandle?

A: When building reusable component libraries where you want to expose a controlled API (focus, scroll, reset) without giving the parent access to the full DOM node. Common in design systems and form components.

#9

useLayoutEffect

Like `useEffect` but fires synchronously BEFORE the browser paints.

What is `useLayoutEffect`?

In plain English: **`useLayoutEffect`** has the same signature as `useEffect`, but it runs at a different time. `useEffect` runs AFTER the browser has painted the screen. `useLayoutEffect` runs AFTER React calculates the DOM changes but BEFORE the browser paints. This means you can read layout (dimensions, positions) and make synchronous DOM changes without the user seeing a flash.

Real-World Analogy

*`useEffect` is like fixing a typo on a billboard AFTER it's been revealed to the public (they see the flash).
`useLayoutEffect` is like fixing it backstage BEFORE the curtain opens — nobody sees the mistake.*

Timeline

```
// Render timeline:  
// 1. React calculates DOM changes (render phase)  
// 2. React applies changes to DOM (commit phase)  
// 3. useLayoutEffect runs (SYNC — blocks paint)  
// 4. Browser paints the screen  
// 5. useEffect runs (ASYNC — after paint)
```

When to Use — Preventing Flash

```
function Tooltip({ anchorRef, children }) {  
  const tooltipRef = useRef(null);  
  const [position, setPosition] = useState({ top: 0, left: 0 });  
  // useLayoutEffect — measure BEFORE paint → no visual jump  
  useLayoutEffect(() => {  
    const anchor = anchorRef.current.getBoundingClientRect();  
    const tooltip = tooltipRef.current.getBoundingClientRect();  
    setPosition({  
      top: anchor.top - tooltip.height - 8,  
      left: anchor.left + (anchor.width - tooltip.width) / 2,  
    });  
  }, []);  
  return (  
    <div ref={tooltipRef} style={{ position: "fixed", ...position }}>
```

```
{children}
</div>
);
}

// With useEffect: tooltip appears at (0,0) then JUMPS to correct spot
// With useLayoutEffect: tooltip appears directly at correct spot
```

■■ **Default to `useEffect`.** Only switch to `useLayoutEffect` if you see a visual flicker/jump caused by DOM measurements. `useLayoutEffect` blocks painting — overuse causes jank.

Interview Q&A

Q: `useEffect` vs `useLayoutEffect` — summarize in one sentence each.

A: `useEffect`: runs AFTER paint — use for data fetching, subscriptions, anything that doesn't need to block the visual update. `useLayoutEffect`: runs BEFORE paint — use for DOM measurements and mutations where you must avoid a visual flash.

Q: What happens if you use `useLayoutEffect` in SSR (Next.js)?

A: React warns because `useLayoutEffect` can't run on the server (no DOM). Use `useEffect` instead, or conditionally check `typeof window !== 'undefined'`. In Next.js with App Router, this only matters in Client Components.

#10

useDebugValue

Add a label to your custom hook in React DevTools — for debugging only.

What is useDebugValue?

In plain English: **useDebugValue** adds a visible label to your custom hook in React DevTools. It doesn't affect behavior at all — it's purely a developer experience improvement. When you inspect a component in DevTools, you'll see the label next to your custom hook.

Real-World Analogy

It's like putting name labels on cables behind your TV. The cables work the same without labels, but when you need to debug which cable goes where, the labels save you time.

Syntax & Usage

```
function useOnlineStatus() {
  const [isOnline, setIsOnline] = useState(true);
  useEffect(() => {
    const handle = () => setIsOnline(navigator.onLine);
    window.addEventListener("online", handle);
    window.addEventListener("offline", handle);
    return () => {
      window.removeEventListener("online", handle);
      window.removeEventListener("offline", handle);
    };
  }, []);
  // Shows "OnlineStatus: Online" or "OnlineStatus: Offline" in DevTools
  useDebugValue(isOnline ? "Online" : "Offline");
  return isOnline;
}
```

Deferred Formatting (Performance)

If formatting the debug value is expensive, pass a formatter function as the second argument. React will only call it when DevTools is actually open — not on every render.

```
function useData(query) {
  const [data, setData] = useState(null);
  // ...fetch logic...
  // Formatter only runs when DevTools inspects this hook
  useDebugValue(data, (d) => {
```



```
return d ? `${d.items.length} items loaded` : "Loading...";
});
return data;
}
```

■ **When to use:** Only in custom hooks that are shared across your team or published as a library. Don't add `useDebugValue` to every hook — it's for hooks where the internal state is non-obvious.

Interview Q&A

Q: Does `useDebugValue` affect performance or behavior?

A: No. It's completely stripped in production builds and only shows labels in React DevTools during development. It has zero runtime cost in production.

Q: Can you use `useDebugValue` in a regular component (not a custom hook)?

A: Technically you can call it, but it only makes sense in custom hooks. In a component, you can just inspect props and state directly in DevTools.

Quick Reference Cheat Sheet

Hook	Purpose	Triggers Re-render?	Key Gotcha
useState	Add state to component	Yes	Use functional updates for dependent state
useEffect	Side effects after render	No (causes them)	Always clean up timers & subscriptions
useMemo	Cache expensive computation	No	It's a hint, not a guarantee
useCallback	Cache function reference	No	Useless without React.memo on child
useContext	Read context value	Yes (on change)	Re-renders ALL consumers on any change
useReducer	Complex state with actions	Yes	Reducer must be a pure function
useRef	Mutable ref, DOM access	No	Don't update .current during render
useImperativeHandle	Custom ref API	No	Requires forwardRef (pre React 19)
useLayoutEffect	Sync effect before paint	No (causes them)	Blocks paint — use sparingly
useDebugValue	DevTools label	No	Only in custom hooks; stripped in prod

Rules of Hooks (Never Break These)

- **1. Only call hooks at the top level** — never inside if/else, loops, or nested functions.
- 2. Only call hooks from React functions** — function components or custom hooks (use_____).
- 3. Custom hooks must start with 'use'** — useAuth, useFetch, useDebounce, etc.